

Практическое занятие 27: Создание и обработка форм — Реализация отзывов без авторизации

Цель занятия:

- Научиться создавать и обрабатывать формы в Django.
 - Реализовать возможность добавления отзывов к мероприятиям.
 - Понять принципы работы с формами и валидацией данных.
-

Задачи занятия

1. Введение в формы Django
 2. Создание модели `Review`
 3. Создание формы `ReviewForm`
 4. Обновление представления `event_detail` для обработки формы
 5. Обновление шаблона `event_detail.html` для отображения формы и отзывов
 6. Выполнение практического упражнения. Добавление отзывов к мероприятиям
 7. Выполнение дополнительных заданий. Реализация капчи, добавление сообщения об успешной отправке отзыва
-

Задача 1: Введение в формы Django

1.1. Что такое формы в Django?

- **Формы** — это способ взаимодействия пользователей с вашим веб-приложением.
- Позволяют собирать данные от пользователей и обрабатывать их на сервере.
- Django предоставляет встроенную поддержку для работы с формами, включая валидацию и защиту от CSRF-атак.

1.2. Почему важно уметь работать с формами?

- **Взаимодействие с пользователями:** Формы — основной способ ввода данных пользователями.

- **Валидация данных:** Обеспечивает корректность и безопасность введенных данных.
 - **Улучшение UX:** Правильно реализованные формы делают приложение более удобным для пользователей.
-

Задача 2: Создание модели `Review`

2.1. Определение модели `Review`

Шаги:

1. Откройте файл `events/models.py`.
2. Добавьте класс `Review`:

```
# events/models.py

from django.db import models

class Review(models.Model):
    event = models.ForeignKey('Event', related_name='reviews',
on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    email = models.EmailField()
    rating = models.PositiveSmallIntegerField(choices=[(i, i) for i in
range(1, 6)])
    comment = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Review by {self.name} for {self.event.title}'
```

Пояснение:

- `event`: Связь с моделью `Event`, к которой относится отзыв.
- `name`: Имя автора отзыва.
- `email`: Электронная почта автора (для обратной связи, если потребуется).
- `rating`: Оценка мероприятия (от 1 до 5).
- `comment`: Текст отзыва.
- `created_at`: Дата и время создания отзыва.

2.2. Применение миграций

Шаги:

1. Создайте миграции:

```
python manage.py makemigrations
```

2. Примените миграции:

```
python manage.py migrate
```

Задача 3: Создание формы `ReviewForm`

3.1. Создание файла `forms.py`

Если у вас уже есть файл `forms.py` в приложении `events`, пропустите этот шаг.

1. Создайте файл `forms.py` в директории `events/`.

3.2. Определение формы `ReviewForm`

Шаги:

1. Добавьте следующий код в `forms.py`:

```
# events/forms.py

from django import forms
from .models import Review

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }
```

Пояснение:

- Наследуемся от `forms.ModelForm` для автоматического создания полей на основе модели.
- `Meta` класс указывает модель и поля, которые будут использоваться в форме.

- `widgets` позволяет задать виджеты для полей. Для `rating` используем выпадающий список.

Задача 4: Обновление представления `event_detail` для обработки формы

4.1. Обновление функции `event_detail`

Шаги:

1. Откройте файл `views.py` в приложении `events`.
2. Импортируйте `ReviewForm`:

```
# events/views.py

from .forms import ReviewForm
```

3. Обновите функцию `event_detail`:

```
# events/views.py

from django.shortcuts import render, get_object_or_404, redirect
from django.contrib import messages

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':
        review_form = ReviewForm(request.POST)
        if review_form.is_valid():
            review = review_form.save(commit=False)
            review.event = event
            review.save()
            messages.success(request, 'Ваш отзыв успешно добавлен.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            messages.error(request, 'Пожалуйста, исправьте
ошибки в форме.')
    else:
        review_form = ReviewForm()

    return render(request, 'events/event_detail.html', {
        'event': event,
```

```
        'reviews': reviews,
        'review_form': review_form,
    })
```

Пояснение:

- Обработываем метод `POST` для добавления нового отзыва.
- Создаем экземпляр формы `ReviewForm` с данными из запроса.
- Проверяем валидность формы и сохраняем отзыв, связывая его с текущим мероприятием.
- Перенаправляем пользователя на ту же страницу после успешного добавления отзыва.
- Если метод `GET`, отображаем пустую форму для заполнения.

Задача 5: Обновление шаблона `event_detail.html` для отображения формы и отзывов

5.1. Отображение отзывов

Шаги:

1. Откройте файл `event_detail.html` в директории `templates/events/`.
2. Обновите раздел отображения отзывов:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <p><strong>Дата и время:</strong> {{ event.date|date:"d.m.Y H:i" }}</p>
    <p><strong>Место:</strong> {{ event.location }}</p>
    <p><strong>Категория:</strong> {{ event.category.name }}</p>
    <p>{{ event.description }}</p>

    {% if event.main_image %}
        
    {% endif %}
</block>
```

```

{% if event.document %}
    <p><a href="{{ event.document.url }}">Скачать дополнительную
информацию</a></p>
{% endif %}

<h2>Отзывы</h2>
{% if reviews %}
    {% for review in reviews %}
        <div>
            <p><strong>{{ review.name }}</strong> - Оценка: {{
review.rating }}/5</p>
            <p>{{ review.comment }}</p>
            <p><em>{{ review.created_at|date:"d.m.Y H:i" }}</em>
</p>
        </div>
    {% endfor %}
{% else %}
    <p>Нет отзывов о данном мероприятии.</p>
{% endif %}

<!-- Форма добавления отзыва -->
<h2>Оставить отзыв</h2>
<form method="post">
    {% csrf_token %}
    {{ review_form.non_field_errors }}
    {{ review_form.as_p }}
    <button type="submit">Отправить</button>
</form>

<!-- Ссылки для редактирования и удаления (если необходимо) -->
<!-- ... ваш существующий код ... -->
{% endblock %}

```

Пояснение:

- Отображаем список отзывов, используя переменную `reviews`.
- Добавляем форму для добавления нового отзыва, используя `{{ review_form.as_p }}`.

5.2. Отображение сообщений

Шаги:

1. В базовый шаблон `base.html` добавьте отображение сообщений:

```
<!-- templates/base.html -->
```

```
{% if messages %}
    <ul class="messages">
        {% for message in messages %}
            <li{% if message.tags %} class="{{ message.tags }}"{% endif
            %}>{{ message }}</li>
        {% endfor %}
    </ul>
{% endif %}
```

Пояснение:

- Используем фреймворк сообщений **Django** для отображения уведомлений пользователю (например, об успешном добавлении отзыва).

Задача 6: Практическое упражнение: Добавление отзывов к мероприятиям

Задание для студентов:

- Перейдите на страницу любого мероприятия на сайте.
- Добавьте отзыв, заполнив форму.
- Проверьте, что ваш отзыв отображается в списке отзывов для данного мероприятия.
- Попробуйте добавить несколько отзывов с разными данными.

Цель упражнения:

- Закрепить навыки работы с формами в Django.
- Убедиться в корректной работе добавления и отображения отзывов.

Дополнительное задание

1. Добавление капчи для защиты от спама

Цель: Защитить форму от автоматической отправки спам-ботами.

Шаги:

1. Установите пакет `django-simple-captcha`:

```
pip install django-simple-captcha
```

2. Добавьте `captcha` в `INSTALLED_APPS` в `settings.py`:

```
# settings.py

INSTALLED_APPS = [
    # ... другие приложения ...
    'captcha',
]
```

3. Добавьте маршрут для капчи в `urls.py` вашего проекта:

```
# your_project/urls.py

from django.urls import path, include

urlpatterns = [
    # ... другие маршруты ...
    path('captcha/', include('captcha.urls')),
]
```

4. Обновите `ReviewForm`, добавив поле капчи:

```
# events/forms.py

from captcha.fields import CaptchaField

class ReviewForm(forms.ModelForm):
    captcha = CaptchaField()

    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment', 'captcha']
        widgets = {
            'rating': forms.Select(choices=[(i, i) for i in range(1,
6)]),
        }
```

5. Примените миграции для капчи:

```
python manage.py migrate
```

6. Проверьте работу формы, убедившись, что капча отображается и проверяется

при отправке.

2. Добавление сообщения об успешном добавлении отзыва

Цель: Улучшить пользовательский интерфейс, показывая пользователю подтверждение об успешном добавлении отзыва.

Шаги:

1. Убедитесь, что в представлении `event_detail` используется `messages.success` при успешном добавлении отзыва:

```
# events/views.py

messages.success(request, 'Ваш отзыв успешно добавлен.')
```

2. Убедитесь, что в шаблоне `base.html` отображаются сообщения, как показано ранее.
3. Проверьте, что сообщение отображается после добавления отзыва.

Заключение

Поздравляем! Вы успешно:

- Научились создавать и обрабатывать формы в Django.
- Реализовали возможность добавления отзывов к мероприятиям.
- Поняли, как работать с моделями, формами и шаблонами для взаимодействия с пользователем.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- Почему важно проверять валидность формы перед сохранением данных?

- Чтобы обеспечить корректность и безопасность данных, предотвратить ошибки и некорректные записи в базе данных.
 - **Как можно улучшить форму отзыва для лучшего пользовательского опыта?**
 - Добавить интерактивные элементы, использовать AJAX для отправки формы без перезагрузки страницы, предоставить мгновенную обратную связь при валидации.
 - **Какие методы защиты форм от спама существуют?**
 - Использование капчи, ограничение частоты отправки, проверка на роботизированные запросы, использование скрытых полей (honeypot).
-

Полезные ресурсы

- [Документация Django: Работа с формами](#)
- [Документация Django: Фреймворк сообщений](#)
- [Django Crispy Forms](#) — библиотека для улучшения отображения форм.
- [django-simple-captcha](#) — библиотека для добавления капчи в формы.